

Chương 1

Linux là gì và vì sao bạn nên quan tâm

*Phiên bản tối ưu cho THỰC HÀNH và GHI NHỚ.
Bạn sẽ không chỉ đọc — bạn sẽ gõ, sẽ tự kiểm tra, và sẽ nhớ lâu.*

Lộ trình từ con số 0 tới DevOps senior

Dành cho người dùng macOS chưa biết gì về kỹ thuật

Mục lục

0. Trước khi bắt đầu	3
1 Mục tiêu học tập của chương	3
2 Hệ điều hành là gì? (bắt đầu từ con số 0)	4
3 THỰC HÀNH NGAY — Lab 1: Gõ lệnh đầu tiên trong đời	4
3.1 Mở Terminal	5
3.2 Bẫy lệnh đầu tiên	5
3.2.1 Lệnh 1 — <code>uname -a</code> (lệnh “ngôi sao” của chương này)	5
3.2.2 Lệnh 2 — <code>sw_vers</code>	6
3.2.3 Lệnh 3 — <code>whoami</code>	6
3.2.4 Lệnh 4 — <code>date</code>	7
3.2.5 Lệnh 5 — <code>pwd</code>	7
3.2.6 Lệnh 6 — <code>ls</code>	7
3.2.7 Lệnh 7 — <code>echo "Xin chào Linux"</code>	8
4 Linux là gì? (câu chuyện gốc rễ, kể cho dễ nhớ)	9
4.1 Nhân Linux và chàng sinh viên Linus	9
4.2 GNU — mảnh ghép bị lãng quên	9
4.3 “Mã nguồn mở” — và sự nhầm lẫn kinh điển: TỰ DO $\neq$ MIỄN PHÍ	9
5 Sáu thuật ngữ nền tảng (gặp lại suốt cả cuốn sách)	10
6 Linux, macOS, Windows — giống và khác (đây là phần dễ nhầm, đọc kỹ)	11
6.1 Bức tranh tổng quát	11
6.2 macOS chính thức là UNIX — không phải “na ná”	12
6.3 “Giống nhau nhiều” KHÔNG có nghĩa là “y hệt” — cảnh báo cực quan trọng	12
6.4 Bảng so sánh nhanh	13
7 Vì sao người dùng Mac có LỢI THẾ khi học Linux	14
8 Vì sao bạn NÊN học Linux (lý do thực dụng, thẳng thắn)	15
9 Những hiểu lầm thường gặp (đập tan ngay)	16
10 Lộ trình các chương sau (bạn đang đứng ở đâu trên bản đồ)	16
11 Thẻ ghi nhớ (Flashcards)	17
12 Kế hoạch ôn ngắt quãng (Spaced Repetition)	18
13 Đáp án Tự kiểm tra	18
14 Tóm tắt 1 trang	20

15 Bài tập mở rộng (Stretch)

20

0. Trước khi bắt đầu — đọc cái này 30 giây thôi

Nếu bạn chưa từng viết một dòng lệnh nào trong đời, **đúng người rồi đó**. Cuốn này viết cho bạn — người dùng laptop macOS, chưa biết gì về kỹ thuật, nhưng muốn đi từ con số 0 tới trình độ **DevOps senior** (kỹ sư vận hành hệ thống cấp cao).

Đừng để cụm “DevOps senior” làm bạn choáng

Bạn *không* cần biết DevOps là gì lúc này — tôi sẽ giải thích đầy đủ ở mục 8. Ngay bây giờ, mục tiêu duy nhất của bạn là gõ được lệnh đầu tiên trong đời. Con đường tới senior dài, nhưng nó được xây bằng những viên gạch rất nhỏ. Chương này là viên gạch đầu tiên — chỉ cần đặt xong viên gạch này thôi.

Một bí mật nhỏ mà ít người nói với bạn: **chiếc Mac bạn đang dùng đã là một hệ Unix thực thụ** (chút nữa tôi sẽ chứng minh bằng một dòng lệnh, bạn tự gõ và tự thấy). Nghĩa là bạn không cần cài gì cả để bắt đầu. Sân tập đã nằm sẵn trong máy bạn.

Cách dùng chương này cho hiệu quả

Đừng chỉ đọc. Hãy mở Terminal lên và **gõ lại từng lệnh**. Đọc mà không gõ thì như xem video bơi mà không xuống nước. Ghi nhớ đến tận ngón tay, không chỉ từ mắt.

1. Mục tiêu học tập của chương

Sau khi học xong chương này, bạn sẽ **LÀM ĐƯỢC** những việc sau (không phải “biết về”, mà là “làm được”):

1. **Mở được Terminal** trên máy Mac và gõ được lệnh đầu tiên trong đời mà không sợ làm hỏng máy.
2. **Giải thích bằng lời của chính mình** hệ điều hành là gì, Linux là gì, và vì sao nó quan trọng.
3. **Chứng minh được** chiếc Mac của bạn đang chạy trên nền Unix — bằng một lệnh cụ thể, tự tay gõ.
4. **Phân biệt** được Linux, macOS, Windows — cái nào giống cái nào, giống ở đâu, khác ở đâu.
5. **Định nghĩa được** 6 thuật ngữ nền tảng: *kernel*, *distro*, *shell*, *terminal*, *open source*, *Unix-like* — mỗi cái một câu.
6. **Hiểu vì sao** gần như toàn bộ máy chủ (server), điện toán đám mây (cloud) và DevOps đều chạy Linux — và vì sao việc bạn dùng Mac là một lợi thế lớn để học.

Nếu cuối chương bạn làm được 6 việc trên, bạn đã sẵn sàng cho Chương 2. Ta đi nào.

2. Hệ điều hành là gì? (bắt đầu từ con số 0)

Hãy tưởng tượng chiếc laptop của bạn là một tòa nhà văn phòng lớn. Bên trong có **phần cứng** (hardware): màn hình, bàn phím, con chip xử lý (CPU), bộ nhớ (RAM), ổ cứng lưu dữ liệu. Những thứ này là gạch, bê tông, dây điện — bản thân chúng không tự làm được gì cả.

Hệ điều hành (operating system, viết tắt **OS**) là **người quản lý tòa nhà**. Nó là phần mềm đứng giữa bạn và phần cứng, lo mọi việc để bạn không phải lo:

- Khi bạn mở một ứng dụng, OS quyết định cấp cho nó bao nhiêu bộ nhớ.
- Khi bạn lưu một file, OS biết phải ghi nó vào chỗ nào trên ổ cứng.
- Khi hai ứng dụng cùng muốn dùng CPU, OS phân chia thời gian cho công bằng.

Định nghĩa — Hệ điều hành (OS)

Phần mềm nền tảng quản lý phần cứng và làm cầu nối để các ứng dụng (và bạn) sử dụng máy tính.

Những hệ điều hành phổ biến bạn đã nghe tên: **Windows** (của Microsoft), **macOS** (của Apple, đang chạy trên máy bạn), **Linux**, **Android** (điện thoại), **iOS** (iPhone). Tất cả đều là OS.

Góc Senior

Người mới thường nghĩ “máy tính = ứng dụng tôi nhìn thấy”. Senior nghĩ theo **lớp** (layers): phần cứng ở dưới cùng → **nhân hệ điều hành** ở giữa → ứng dụng ở trên. Vì sao quan trọng? Hãy hình dung một tình huống production thật: lúc 2 giờ sáng có cảnh báo “CPU server 100%”. Câu hỏi đầu tiên của senior không phải “restart máy đi” mà là “100% đó là do lớp nào?” — do một **ứng dụng** chạy loạn (sử ở lớp ứng dụng), do **nhân** đang bận xử lý vào/ra ổ đĩa (vấn đề khác hẳn), hay do **phần cứng** sắp hỏng? Biết hỏi đúng lớp giúp bạn sửa trúng chỗ thay vì restart mù quáng rồi lỗi lại tái diễn. “Tư duy theo lớp” là tư duy nền tảng của mọi kỹ sư hệ thống. Ta sẽ quay lại ý này suốt cả cuốn sách.

Tự kiểm tra 2

(chưa xem đáp án vội — tự trả lời trong đầu hoặc viết ra giấy)

1. Bằng lời của bạn, hệ điều hành làm nhiệm vụ gì giữa bạn và phần cứng?
2. Kể tên 3 hệ điều hành. Máy bạn đang dùng cái nào?
3. “Tư duy theo lớp” gồm những lớp nào, từ dưới lên trên?

3. THỰC HÀNH NGAY — Lab 1: Gõ lệnh đầu tiên trong đời

Đây là phần quan trọng nhất của chương. **Hãy làm thật**, đừng đọc lướt.

3.1 Mở Terminal

Terminal là một ứng dụng có sẵn trong mọi máy Mac. Nó là một **cửa sổ để bạn gõ lệnh bằng chữ** thay vì bấm chuột. (Định nghĩa kỹ hơn ở mục 5.)

Cách mở nhanh nhất bằng **Spotlight** (công cụ tìm kiếm của Mac):

1. Bấm tổ hợp phím **Command (phím Cmd) + Space (phím cách)**. Một ô tìm kiếm hiện ra giữa màn hình.
2. Gõ chữ: **Terminal**
3. Bấm **Enter**.

Một cửa sổ nền (thường màu trắng hoặc đen) hiện ra, với một dòng chữ kết thúc bằng dấu **%** rồi một con trỏ nhấp nháy. Đó gọi là **dấu nhắc lệnh** (prompt) — máy đang nói: “*Tôi sẵn sàng, mời bạn gõ.*”

Quy ước trong sách

Khi tôi viết một lệnh, bạn **chỉ gõ phần lệnh**, KHÔNG gõ dấu **%** (đó là máy tự in ra). Gõ xong mỗi lệnh thì bấm **Enter** để chạy.

An toàn tuyệt đối

Tất cả lệnh trong chương này chỉ **đọc thông tin** từ máy và in ra màn hình. Không lệnh nào xóa, sửa hay làm hỏng bất cứ thứ gì. Bạn cứ gõ thoải mái.

3.2 Bẫy lệnh đầu tiên

Gõ từng lệnh, bấm Enter, **quan sát kết quả rồi mới đọc phần giải thích**.

Lệnh 1 — `uname -a` (lệnh “ngôi sao” của chương này)

Lệnh

```
uname -a
```

Kết quả mong đợi (trên máy mẫu — máy bạn sẽ hơi khác về tên và số, đó là chuyện bình thường):

```
Darwin macbook25.4.0 Darwin Kernel Version 25.4.0: ...;
root:xnu-12377.101.15~1/RELEASE_ARM64_T6041 arm64
```

Ý nghĩa — và đây là khoảnh khắc “aha”: `uname` là viết tắt của *Unix name* (tên hệ Unix). Cờ `-a` nghĩa là *all* — in ra tất cả thông tin.

Hãy nhìn chữ đầu tiên: **Darwin**. Darwin là tên **nhân (kernel)** của macOS. Việc lệnh `uname` — một lệnh sinh ra từ thế giới Unix — chạy được và in ra thông tin chính là **bằng chứng tận mắt rằng máy Mac của bạn là một hệ Unix**. Bạn không cần tin lời tôi; máy bạn vừa tự khai báo điều đó.

Hai chi tiết nữa đáng để ý ngay trong dòng kết quả:

- Chuỗi **xnu-12377...** ở gần cuối chính là tên đầy đủ của nhân macOS: **XNU**. Hãy nhớ chuỗi này — đến mục 6.2 ta sẽ khẳng định “nhân macOS tên đúng là XNU”, và đây là bằng chứng nhìn-thấy-được ngay trên máy bạn.
- Chữ **arm64** ở cuối cho biết máy bạn dùng chip Apple Silicon (M1/M2/M3/M4...).

Hãy dừng lại 5 giây và cảm nhận điều này

Bạn vừa dùng đúng một lệnh mà các kỹ sư Linux dùng hằng ngày trên server, và nó chạy ngay trên Mac của bạn. Cây cầu nối giữa bạn và thế giới Linux vừa được bắc xong.

Lệnh 2 — `sw_vers`

Lệnh

```
sw_vers
```

Kết quả mong đợi (trên máy mẫu):

```
ProductName:      macOS
ProductVersion:   26.4.1
BuildVersion:     25E253
```

Đừng lo nếu số khác

Số phiên bản trên máy bạn gần như chắc chắn khác con số mẫu này — điều đó hoàn toàn bình thường, mỗi máy chạy một phiên bản macOS khác nhau. Bạn không gõ nhầm gì cả.

Ý nghĩa: `sw_vers` là viết tắt của *software version*. Đây là lệnh **riêng của macOS** (không có trên Linux), cho biết bạn đang chạy macOS phiên bản nào. Ghi nhớ điểm này: có lệnh dùng chung cả hai bên (như `uname`), có lệnh chỉ riêng một bên (như `sw_vers`). Ta sẽ bàn kỹ ở mục 6.

Lệnh 3 — `whoami`

Lệnh

```
whoami
```

Kết quả mong đợi: tên tài khoản của bạn, ví dụ:

```
vinh
```

Ý nghĩa: “Tôi là ai?” — máy in ra **tên người dùng (user)** mà bạn đang đăng nhập. Trong thế giới Unix/Linux, *mọi thứ đều gắn với một người dùng*: ai được làm gì, ai sở hữu file nào. Khái niệm “user” này về sau cực kỳ quan trọng (phân quyền, bảo mật). Bạn vừa gặp nó lần đầu.

Lệnh 4 — date

Lệnh

```
date
```

Kết quả mong đợi:

```
Fri Jun 26 22:23:28 +07 2026
```

Đương nhiên máy bạn in ra ngày giờ hiện tại của chính bạn

Đó là điều hiển nhiên, vì `date` luôn lấy thời gian thực. Miễn là có một dòng ngày giờ hiện ra là đúng.

Ý nghĩa: in ngày giờ hệ thống cùng **múi giờ** (+07 là giờ Việt Nam). Nghe đơn giản, nhưng trên server thật, sai múi giờ là nguyên nhân kinh điển của hàng loạt lỗi khó hiểu. (Xem Góc Senior bên dưới.)

Lệnh 5 — pwd

Lệnh

```
pwd
```

Kết quả mong đợi: một đường dẫn, ví dụ:

```
/Users/vinh
```

Ý nghĩa: `pwd` = *print working directory* (in ra thư mục hiện hành). Nó trả lời câu “**Tôi đang đứng ở đâu?**”. Khi dùng Terminal, bạn luôn “đứng” ở một thư mục nào đó, y như khi mở Finder bạn luôn đang ở trong một folder. Dấu / ngăn cách các thư mục lồng vào nhau (folder nằm trong folder — giống y như trong Finder; cách sắp xếp lồng nhau này về sau gọi là **cây thư mục**, ta sẽ học kỹ ở Chương 2).

Lệnh 6 — ls

Lệnh

```
ls
```

Kết quả mong đợi: danh sách tên các file và thư mục tại chỗ bạn đang đứng, ví dụ:

```
Desktop  Documents  Downloads  Music  Pictures
```

Danh sách trên máy bạn gần như chắc chắn sẽ khác

Nhiều hay ít thư mục hơn đều bình thường, tùy bạn có những gì trong máy (nhiều người còn thấy thêm `Library`, `Movies`, `Public...`). Miễn là có một danh sách hiện ra là đúng.

Ý nghĩa: `ls` = *list* (liệt kê). Đây là lệnh bạn sẽ gõ nhiều nhất trong đời làm việc với dòng lệnh. Nó cho bạn “nhìn” được những gì đang có trong thư mục hiện hành — giống như mở một folder trong Finder để xem bên trong.

Lệnh 7 — `echo "Xin chào Linux"`

Lệnh

```
echo "Xin chào Linux"
```

Kết quả mong đợi:

```
Xin chào Linux
```

Ý nghĩa: `echo` nghĩa là “in ra màn hình đúng những gì tôi đưa cho”. Nghe vô dụng? Thực ra `echo` là viên gạch nền của **kịch bản tự động** (script) — về sau bạn sẽ dùng nó để máy tự “nói” trạng thái, ghi nhật ký, thông báo. Hôm nay nó chỉ chào Linux thay bạn thôi.

Góc Senior — vì sao `date` lại quan trọng đến thế?

Trên một hệ thống production thật, hàng chục server phải đồng bộ thời gian với nhau. Nếu một server lệch giờ, thì: log (nhật ký) ghi sai thứ tự sự kiện khiến bạn điều tra sự cố trong vô vọng; chứng chỉ bảo mật (certificate) có thể bị coi là “hết hạn” hoặc “chưa hiệu lực”; các tác vụ hẹn giờ (cron) chạy sai lúc. Senior luôn kiểm tra `date` và múi giờ như một phản xạ khi đụng vào một máy lạ. Bài học: **đừng bao giờ coi thường một lệnh “trông có vẻ tầm thường”**. Trong vận hành, cái nhỏ nhất thường gây ra cái lớn nhất.

Cạm bẫy thường gặp

Gõ sai một chữ và máy báo `command not found`? Đừng hoảng. Đó **không phải** lỗi của máy bị hỏng — chỉ là bạn gõ một từ mà máy không hiểu. Gõ lại cho đúng chính tả là xong. Người mới hay sợ “lỡ làm hỏng máy”; thực tế các lệnh đọc thông tin như trên thì gõ sai cũng vô hại, máy chỉ từ chối làm. Cứ mạnh dạn.

Tự kiểm tra 3

1. Lệnh nào chứng minh máy Mac của bạn là một hệ Unix, và **chữ nào** trong kết quả là bằng chứng?
2. Trong các lệnh vừa học, lệnh nào **chỉ có riêng** trên macOS chứ không có trên Linux?
3. `pwd` và `ls` khác nhau ở chỗ nào? (gợi ý: một cái trả lời “ở đâu”, một cái trả lời “có gì”)

4. Linux là gì? (câu chuyện gốc rễ, kể cho dễ nhớ)

Bây giờ bạn đã chạm tay vào Unix rồi, ta nói về **Linux** — nhân vật chính của cuốn sách.

4.1 Nhân Linux và chàng sinh viên Linus

Năm **1991**, một sinh viên người Phần Lan tên **Linus Torvalds** viết một thứ gọi là **nhân (kernel)** cho vui, rồi chia sẻ miễn phí lên Internet để mọi người cùng nghịch. Anh đặt tên nó là **Linux** (ghép từ tên Linus + Unix). Không ngờ, hàng nghìn lập trình viên trên khắp thế giới nhảy vào cùng cải tiến. Hơn 30 năm sau, **Linux trở thành hệ điều hành chạy phần lớn Internet** — từ Google, Facebook, Netflix cho tới gần như mọi máy chủ đám mây.

Định nghĩa — Nhân (kernel)

Phần lõi nhất, sâu nhất của hệ điều hành. Nó là thứ trực tiếp nói chuyện với phần cứng và quản lý bộ nhớ, CPU, thiết bị. Hãy nhớ: **“Linux” — về mặt chính xác kỹ thuật — chỉ là cái nhân này**. Phần còn lại (cửa sổ, ứng dụng, lệnh) được ghép thêm vào.

4.2 GNU — mảnh ghép bị lãng quên

Câu chuyện chưa trọn nếu thiếu một nhân vật trước đó. Năm **1983**, **Richard Stallman** khởi xướng dự án **GNU** với một giấc mơ: tạo ra một hệ điều hành **hoàn toàn tự do**, ai cũng được xem mã nguồn, sửa, và chia sẻ. Đến năm 1991, GNU đã có gần đủ mọi công cụ của một hệ điều hành — chỉ còn **thiếu đúng cái nhân**. Và Linux của Linus xuất hiện đúng lúc, lấp vào chỗ trống đó.

Vì vậy, nói cho thật chính xác, cái mà người ta quen gọi là “Linux” thực ra là sự kết hợp **GNU + Linux** (nhiều người gọi đầy đủ là *GNU/Linux*): công cụ GNU mặc lên nhân Linux. Bạn không cần thuộc lòng điều này, nhưng nên biết để hiểu vì sao về sau ta hay nhắc tới “công cụ GNU”.

4.3 “Mã nguồn mở” — và sự nhầm lẫn kinh điển: TỰ DO ≠ MIỄN PHÍ

Mã nguồn mở (open source) nghĩa là: phần mã (công thức) làm nên phần mềm được **công khai** — ai cũng xem được, sửa được, chia sẻ được.

Đây là điểm gần như **ai mới học cũng hiểu nhầm**, hãy đọc kỹ:

- Trong tiếng Anh, chữ **“free”** có hai nghĩa: *tự do* và *miễn phí*. Triết lý của Linux/GNU nói tới **“free” = TỰ DO**, không phải “free = miễn phí”.
- Một câu nói kinh điển để nhớ: *“free as in **freedom** (tự do ngôn luận), not free as in **beer** (bia miễn phí).”* Về “bia miễn phí” nghĩa là: ai đó cho bạn một lon bia mà bạn không phải trả tiền — tức đang nói về **GIÁ**. Còn “tự do ngôn luận” nói về **QUYỀN**.

- Nghĩa là: bạn có **quyền tự do** xem mã, sửa, chia sẻ. Việc nó thường cũng miễn phí chỉ là hệ quả phụ. Hoàn toàn có công ty bán Linux có thu tiền (ví dụ Red Hat bán dịch vụ hỗ trợ) — vẫn là mã nguồn mở, vẫn “tự do”, nhưng **không** miễn phí.

Mẹo nhớ

Tự do nói về *quyền của bạn với phần mềm*. **Miễn phí** nói về *giá tiền*. Hai chuyện khác nhau hoàn toàn.

Góc Senior

Vì sao senior quan tâm chuyện “tự do” này? Vì nó là lý do **bạn được phép soi vào tận ruột gan hệ thống**. Khi một server Linux gặp lỗi lạ, bạn có thể đọc thẳng mã nguồn để hiểu nó hoạt động ra sao — điều gần như bất khả thi với phần mềm đóng. “Tự do” không phải triết lý suông; nó là **siêu năng lực gỡ lỗi** của dân DevOps.

Tự kiểm tra 4

1. Ai viết nhân Linux, năm nào? Dự án GNU do ai khởi xướng, năm nào?
2. Nói thật chính xác, “Linux” chỉ là gì — toàn bộ hệ điều hành, hay chỉ một phần?
3. Giải thích sự khác nhau giữa “tự do” và “miễn phí” (có thể mượn ví dụ chai bia, hoặc ví dụ của riêng bạn).

5. Sáu thuật ngữ nền tảng (gặp lại suốt cả cuốn sách)

Ta gom lại và định nghĩa thật gọn 6 từ bạn sẽ gặp đi gặp lại. Đọc chậm, mỗi cái một câu.

Thuật ngữ	Định nghĩa một câu
Kernel (nhân)	Lõi sâu nhất của OS, trực tiếp điều khiển phần cứng (CPU, bộ nhớ, thiết bị).
Distro (bản phân phối)	Một “gói trọn bộ” = nhân Linux + công cụ + ứng dụng, đóng gói sẵn để cài và dùng ngay.
Shell	Chương trình <i>dịch</i> lệnh bạn gõ thành hành động cho hệ điều hành (ví dụ: zsh, bash).
Terminal	Cửa sổ/ứng dụng để bạn <i>nhìn thấy</i> và <i>gõ</i> vào shell.
Open source (mã nguồn mở)	Mã nguồn được công khai để ai cũng xem, sửa, chia sẻ.
Unix-like (giống Unix)	Hệ điều hành theo cùng triết lý và cách làm của Unix (Linux và macOS đều thuộc nhóm này).

Lưu ý từ “dịch” trong định nghĩa Shell

Ý là shell **dịch** thứ bạn gõ (ví dụ chữ `ls`) thành **hành động** mà máy thực hiện (liệt kê file). Bạn gõ bằng “ngôn ngữ người”, shell chuyển nó thành “việc làm của máy”. (Thuật ngữ kỹ thuật gọi việc này là *thông dịch* — interpreter — nhưng bạn chưa cần nhớ từ đó lúc này.)

Hai cặp dễ nhầm nhất, làm rõ ngay:

- **Shell $\neq$ Terminal.** Hãy hình dung: **Terminal là cái điện thoại** (vỏ máy bạn cầm), còn **shell là người ở đầu dây bên kia** thật sự lắng nghe và làm việc. Lúc này khi bạn gõ `ls`, **Terminal** hiển thị chữ, còn **shell** mới là cái hiểu “à, liệt kê file” rồi thực hiện. Trên Mac đời nay, shell mặc định là **zsh** (ta sẽ kiểm chứng ở mục 7).
- **Distro là gì, vì sao cần?** Vì nhân Linux trần trụi thì không dùng trực tiếp được — giống như bạn có động cơ xe nhưng chưa có khung, bánh, vô-lăng. **Distro** là bên ghép động cơ (nhân) với mọi thứ còn lại thành “chiếc xe hoàn chỉnh”. Vài distro phổ biến bạn sẽ nghe suốt:
 - **Ubuntu** — thân thiện người mới, phổ biến nhất, rất nhiều hướng dẫn trên mạng.
 - **Debian** — nền tảng mà Ubuntu xây dựng lên; nổi tiếng ổn định.
 - **Fedora** — đi đầu về công nghệ mới, do cộng đồng quanh Red Hat phát triển.

Mẹo nhớ

Cùng một **nhân Linux**, nhưng đóng gói theo nhiều cách → ra nhiều **distro** khác nhau, giống như cùng một loại bột mì làm ra nhiều loại bánh.

Tự kiểm tra 5

1. Phân biệt **shell** và **terminal** bằng ví dụ “điện thoại / người đầu dây”.
2. Distro là gì? Kể tên 3 distro.
3. Vì sao không ai dùng “nhân Linux trần” trực tiếp mà phải qua một distro?

6. Linux, macOS, Windows — giống và khác (đây là phần dễ nhầm, đọc kỹ)

6.1 Bức tranh tổng quát

Có một “cây gia phả” của các hệ điều hành. Ngày xưa có **Unix** (khởi tạo năm **1969** tại phòng thí nghiệm Bell Labs). Từ Unix, sinh ra hai nhánh tư duy:

- **Nhóm Unix-like** (*giống Unix — định nghĩa kỹ ở mục 5*): gồm **Linux** và **macOS**. Chúng theo cùng triết lý Unix → cách dùng dòng lệnh **rất giống nhau**. Đây chính là lý do Mac của bạn là sân tập tuyệt vời để học Linux.
- **Windows**: đi theo một dòng dõi hoàn toàn khác (nhân NT). Dòng lệnh và cách tổ chức bên trong khác biệt đáng kể.

6.2 macOS chính thức là UNIX — không phải “na ná”

Đây là điểm khiến Mac đặc biệt, và cần nói cho thật chính xác:

- **macOS là một hệ UNIX được chứng nhận chính thức** bởi tổ chức **The Open Group** (tổ chức giữ chuẩn và thương hiệu “UNIX”). Tức là macOS không chỉ “giống Unix” — nó **đạt chuẩn UNIX thật sự**, có giấy chứng nhận hẳn hoi.
- **Nhân của macOS tên là XNU** — chính là chuỗi `xnu-...` bạn đã nhìn thấy trong kết quả `uname -a` ở Lab 1. *Mach* chỉ là một **bộ phận bên trong XNU**, **không phải** một nhánh của Unix.

Đỉnh chính một hiểu lầm hay gặp

Nhiều tài liệu trên mạng nói “*Mach là một nhánh của Unix*” — điều này **không đúng**. Hãy nhớ gọn hai điều: (1) **nhân macOS tên đúng là XNU**; (2) **Mach chỉ là một thành phần bên trong XNU**, **không phải** nhánh/biến thể của Unix. Bạn không cần đi sâu hơn ở giai đoạn này.^a

^a*Ghi chú cho người tò mò (không bắt buộc đọc):* “Mach” là một **microkernel** — một kiểu thiết kế nhân trong đó phần lõi được giữ tối giản — do Đại học Carnegie Mellon nghiên cứu. XNU lắp ghép Mach với các thành phần kiểu BSD. Đây là kiến trúc nâng cao, hoàn toàn có thể bỏ qua ở Chương 1.

Còn **Linux** thì sao? Linux **không** mang chứng nhận UNIX của The Open Group, mà được xếp vào nhóm “**Unix-like**” — *xây dựng lại theo tinh thần và chuẩn giao diện của Unix* mà không dùng mã gốc của Unix. Kết quả thực dụng cho bạn: **cách gõ lệnh của Linux và macOS giống nhau đến mức ngạc nhiên**.

6.3 “Giống nhau nhiều” KHÔNG có nghĩa là “y hệt” — cảnh báo cực quan trọng

Đây là cái bẫy số một của người học Linux trên Mac, nên tôi phải nói rõ:

- Rất nhiều lệnh **trùng tên và dùng gần như y nhau** trên cả hai: `ls`, `cd`, `pwd`, `cat`, `cp`, `mv`...
- Nhưng các **tùy chọn (flag/option)** đôi khi khác nhau. Lý do: phần lớn **Linux dùng công cụ phiên bản GNU**, còn **macOS dùng công cụ phiên bản BSD**. Cùng một lệnh, nhưng “hãng sản xuất” khác nhau → một số cờ hành xử khác.

Định nghĩa — Flag (cờ / tùy chọn)

Một mẫu thêm vào sau lệnh để chỉnh cách nó hoạt động, thường bắt đầu bằng dấu `-`. Ví dụ trong `uname -a`, thì `-a` chính là một flag.

Ví dụ kinh điển 1 — tô màu khi liệt kê file: lệnh `ls` liệt kê file. Muốn nó tô màu cho dễ nhìn:

- Trên **macOS (BSD)**, bạn dùng `ls -G`.
- Trên **Linux (GNU)**, bạn lại dùng `ls --color=auto` (cú pháp hai gạch `--color`).

Cùng mục đích, khác cờ.

Ví dụ kinh điển 2 — cộng/trừ ngày tháng: muốn lấy “ngày mai” (cộng 1 ngày vào hôm nay):

- Trên **macOS (BSD)**, bạn dùng `date -v +1d` (chữ `-v` của BSD dùng để cộng/trừ thời gian; `+1d` = cộng 1 ngày). Lệnh này chạy thật trên máy mẫu và in ra ngày của hôm sau.
- Trên **Linux (GNU)**, bạn lại dùng `date -d "tomorrow"` hoặc `date -d "+1 day"`.

Lại đúng kiểu **cùng mục đích, khác cờ** — và đây là ví dụ đã được kiểm chứng chạy được trên cả hai loại máy.

Góc Senior — cạm bẫy “viết trên Mac, chạy trên Linux thì hỏng”

Một sai lầm rất hay gặp của người mới (và cả người không-mới chủ quan): viết một script trên Mac, chạy ngon lành, rồi đẩy lên server Linux thì **lỗi tùm lum** vì các cờ BSD không tồn tại trên GNU (ví dụ đem `date -v +1d` của Mac chạy trên Linux sẽ báo lỗi). Senior xử lý điều này bằng cách: (1) ý thức rõ “máy của tôi là BSD, server là GNU”; (2) khi cần chắc chắn, cài bản GNU trên Mac (qua công cụ Homebrew, ta học sau) hoặc test thẳng trên môi trường Linux (qua Docker/máy ảo, cũng học sau). **Bài học cốt lõi: luôn biết mình đang đứng trên nền nào.** Đây lại là “tư duy theo lớp” ở mục 2 — bạn thấy nó quay lại chưa?

6.4 Bảng so sánh nhanh

	Linux	macOS	Windows
Nhóm	Unix-like	UNIX (chứng nhận chính thức)	Riêng (nhân NT)
Nhân (kernel)	Linux	XNU	NT
Dòng lệnh giống Linux?	(là chuẩn)	Rất giống (khác vài flag BSD vs GNU)	Khác đáng kể
Mã nguồn mở?	Có	Một phần lõi mở (Darwin); phần lớn đóng	Không (đóng)
Hay dùng làm server/cloud?	Cực kỳ phổ biến	Hiếm	Có, ít hơn Linux

Tự kiểm tra 6

1. macOS là “Unix-like” hay “UNIX chính thức”? Ai cấp chứng nhận đó? Còn Linux thuộc nhóm nào?
2. Tên đúng của nhân macOS là gì? *Mach* có phải là một nhánh của Unix không?
3. Vì sao một script viết trên Mac có thể hỏng khi chạy trên Linux? (gợi ý: GNU vs BSD)

7. Vì sao người dùng Mac có LỢI THẾ khi học Linux

Bạn đang cầm một công cụ học tập tuyệt vời mà không phải ai cũng có. Cụ thể:

1. **Terminal sẵn có, không cần cài gì.** Bạn vừa chứng kiến ở Lab 1: mở Terminal là gõ lệnh Unix được ngay. Người dùng Windows phải cài thêm công cụ mới có trải nghiệm tương đương.
2. **Shell mặc định là zsh** — kể từ macOS Catalina (2019) trở đi (trước đó Mac dùng bash). Đây là mốc lịch sử; máy của bạn ngày nay (dù đang chạy phiên bản macOS nào) thì mặc định vẫn là zsh. zsh hiện đại, mạnh, và quan trọng: **các lệnh cơ bản gõ tay** (như `ls`, `cd`, `pwd`) gần như giống hệt trên server Linux, nên kỹ năng dòng lệnh bạn luyện chuyển sang Linux gần như nguyên vẹn.

Hãy tự kiểm chứng — gõ:

Lệnh

```
echo $SHELL
```

Kết quả mong đợi (trên máy mẫu):

```
/bin/zsh
```

Dòng này cho biết **shell đăng nhập mặc định của tài khoản bạn** là zsh (lưu ý: `$SHELL` chỉ ra shell *đăng nhập* mặc định — không nhất thiết là shell đang chạy ngay lúc đó; nếu bạn vừa gõ `bash` để vào một phiên bash, `$SHELL` vẫn in `/bin/zsh`). Muốn biết shell *đang chạy* ngay lúc này thì cần lệnh khác, ta sẽ gặp sau. `$SHELL` là một *biến môi trường* — một “ô nhớ” hệ thống lưu sẵn thông tin; ta học kỹ về biến môi trường ở chương sau.

Lưu ý quan trọng kéo nhằm

Nhiều server Linux mặc định dùng **bash** chứ không phải zsh. Nhưng đừng lo: các lệnh cơ bản gõ tay gần như giống hệt nhau giữa zsh và bash, nên kỹ năng dòng lệnh của bạn vẫn chuyển giao được. Khác biệt giữa zsh và bash chỉ lộ ra khi bạn viết **script nâng cao** (học sau) — lúc đó ta sẽ nói rõ.

3. **Lưu ý quan trọng: Bash trên macOS khá CŨ.** Mac vẫn có sẵn bash, nhưng là một phiên bản đã già. Tự kiểm chứng — gõ:

Lệnh

```
bash --version
```

Kết quả mong đợi (trên máy mẫu):

```
GNU bash, version 3.2.57(1)-release (arm64-apple-darwin25)
```

Thấy chưa? **Bản 3.2** — đây là dòng bash **phát hành từ năm 2006**, tức gần 20 năm trước! (Bạn có thể thấy chữ “Copyright (C) 2007” trong output — nhưng đó là năm *bản quyền của bản build do Apple đóng gói*, không phải năm phát hành của

dòng bash 3.2.) Trong khi Linux ngày nay thường dùng bash 5.x. Vì sao Apple giữ bản cũ? Lý do liên quan tới giấy phép phần mềm (bash 4 trở lên dùng giấy phép GPLv3 mà Apple không muốn kèm vào macOS). Bạn chưa cần hiểu sâu, chỉ cần **nhớ một điều thực dụng**: nếu sau này bạn học các tính năng bash mới mà thấy “trên Mac không chạy”, rất có thể thủ phạm là **bash cũ** — và cách xử lý là cài bản bash mới qua Homebrew (học sau).

Góc Senior

Vì sao senior để tâm tới phiên bản công cụ? Vì **“chạy trên máy tôi” không có nghĩa là “chạy trên server”**. Sự khác biệt phiên bản (bash 3.2 trên Mac vs bash 5 trên server) là nguồn gốc của vô số lỗi “không thể tái hiện”. Phản xạ của người làm nghề: trước khi đổ lỗi cho code, hãy hỏi *“phiên bản công cụ ở hai nơi có giống nhau không?”*. Thói quen gõ `--version` để kiểm tra là một thói quen vàng.

Tự kiểm tra 7

1. Shell mặc định của macOS từ năm 2019 (Catalina) trở đi là gì? Lệnh nào giúp bạn tự kiểm chứng?
2. Bash trên Mac mới hay cũ? Lệnh nào cho bạn biết phiên bản?
3. Vì sao “chạy được trên máy tôi” chưa chắc “chạy được trên server”?

8. Vì sao bạn **NÊN** học Linux (lý do thực dụng, thẳng thắn)

Nếu bạn muốn đi đường dài tới DevOps/cloud, đây là sự thật trần trụi:

- **Gần như toàn bộ máy chủ (server) trên thế giới chạy Linux.** Khi bạn mở một website, gửi một tin nhắn, xem một video — nhiều khả năng phía sau là một (hoặc hàng nghìn) máy Linux đang phục vụ bạn.
- **Điện toán đám mây (cloud) chủ yếu là Linux.** Các nền tảng lớn (AWS, Google Cloud, Azure) đều chạy phần lớn workload trên Linux. Muốn làm cloud → phải biết Linux.
- **DevOps gần như đồng nghĩa với Linux.** Các công cụ trụ cột của nghề — Docker (đóng gói ứng dụng), Kubernetes (điều phối hàng loạt ứng dụng), CI/CD (tự động build và triển khai) — đều **sinh ra từ và sống trên Linux**.
- **Kỹ năng dòng lệnh chuyên giao được và bền.** Giao diện đồ họa thay đổi xoành xoạch, nhưng lệnh `ls`, `cd`, `grep`... đã ổn định hàng chục năm và sẽ còn dùng tiếp hàng chục năm nữa. Bạn học một lần, dùng cả sự nghiệp.

Định nghĩa nhanh — DevOps

Cách làm việc kết hợp *phát triển phần mềm* (Development) và *vận hành hệ thống* (Operations), với mục tiêu đưa phần mềm tới tay người dùng nhanh, ổn định và tự

động. Linux là mặt đất mà toàn bộ ngành này đứng lên. (Đây chính là “DevOps senior” mà mục 0 đã nhắc — giờ bạn đã biết nó là gì rồi.)

9. Những hiểu lầm thường gặp (đập tan ngay)

- **“Linux khó, chỉ dành cho thiên tài.”** → Không. Bạn vừa gõ thành công 7 lệnh ở Lab 1 đây thôi — trong đó **6 lệnh là lệnh Unix dùng chung với Linux** (`uname`, `whoami`, `date`, `pwd`, `ls`, `echo`), chỉ riêng `sw_vers` là lệnh riêng của macOS. Khó hay dễ là chuyện *luyện tập đều đặn*, không phải tài năng bẩm sinh.
- **“Linux = miễn phí nên là đồ rẻ tiền/kém.”** → Sai kép. Thứ nhất, “tự do” ≠ “miễn phí” (mục 4.3). Thứ hai, Linux chạy những hệ thống đắt giá và quan trọng bậc nhất hành tinh.
- **“Phải xóa macOS, cài Linux mới học được.”** → Hoàn toàn không cần (và đừng làm vậy lúc này!). Mac của bạn đã là một hệ Unix — dư sức làm sân tập cho cả chục chương đầu. Về sau ta sẽ dùng thêm máy ảo/Docker để chạy Linux thật, **mà không đụng gì tới macOS của bạn.**
- **“Lệnh trên Mac và Linux y hệt nhau.”** → Gần giống, không y hệt (mục 6.3: GNU vs BSD). Nhớ kỹ kéo dính bẫy.
- **“Dùng dòng lệnh dễ làm hỏng máy.”** → Các lệnh *đọc thông tin* (như cả chương này) thì vô hại. Có một số lệnh *xóa/sửa* cần cẩn thận — và sách sẽ luôn cảnh báo rõ trước khi bạn gặp chúng. Cứ yên tâm tiến bước.

10. Lộ trình các chương sau (bạn đang đứng ở đâu trên bản đồ)

Chương 1 là vạch xuất phát. Đường đi tới “senior” được chia thành các chặng nhỏ, mỗi chặng vẫn theo đúng kiểu **học là làm**:

- **Chương 2 — Hệ thống file & di chuyển:** thành thạo `cd`, `pwd`, `ls`, hiểu cây thư mục, đường dẫn tuyệt đối/tương đối.
- **Chương 3 — Làm việc với file:** xem, tạo, sao chép, di chuyển, xóa file (`cat`, `cp`, `mv`, `rm...`) — và cách làm an toàn.
- **Chương 4 — Người dùng & phân quyền:** ai được làm gì, `chmod`, `sudo`, vì sao phân quyền là xương sống bảo mật.
- **Chương 5 — Shell & script:** biến môi trường, viết script tự động hóa đầu tiên (đây là lúc `echo` tỏa sáng).
- **Các chương sau:** quản lý tiến trình, mạng, gói phần mềm, rồi bước vào Docker, máy chủ, cloud và DevOps.

Mỗi chương đều có **Lab thực hành + Tự kiểm tra + Flashcards + lịch ôn ngắt quãng** y như chương này. Cứ đi đều, bạn sẽ tới.

11. Thẻ ghi nhớ (Flashcards)

Cách dùng

Che cột phải, đọc cột trái, tự đọc to định nghĩa. Đúng thì qua, sai thì đánh dấu để ôn lại.

Thuật ngữ	Định nghĩa 1 dòng
Hệ điều hành (OS)	Phần mềm quản lý phần cứng và làm cầu nối giữa bạn với máy.
Kernel (nhân)	Lõi sâu nhất của OS, trực tiếp điều khiển phần cứng.
Linux	Nhân do Linus Torvalds viết năm 1991; nền tảng của phần lớn server thế giới.
GNU	Dự án của Richard Stallman (1983) tạo công cụ tự do; ghép với nhân Linux thành GNU/Linux.
Open source (mã nguồn mở)	Mã nguồn công khai, ai cũng xem/sửa/chia sẻ được.
Tự do $\neq$ miễn phí	“Free” = quyền tự do với phần mềm, không phải giá bằng 0.
Distro	Gói trọn bộ: nhân Linux + công cụ + ứng dụng (Ubuntu, Debian, Fedora).
Shell	Chương trình dịch lệnh bạn gõ thành hành động cho máy (zsh, bash).
Terminal	Cửa sổ/ứng dụng để bạn gõ và xem shell làm việc.
Unix-like	Hệ theo triết lý Unix (Linux, macOS đều thuộc nhóm này).
UNIX (chứng nhận)	Chuẩn chính thức của The Open Group; macOS đạt chuẩn này .
XNU	Tên đúng của nhân macOS (Mach chỉ là một thành phần bên trong, không phải nhánh Unix).
Flag (cờ/tùy chọn)	Mẫu thêm sau lệnh, thường bắt đầu bằng -, để chỉnh cách lệnh chạy.
GNU vs BSD	Hai “dòng” công cụ: Linux thường dùng GNU, macOS dùng BSD → một số flag khác nhau (vd <code>ls -G BSD</code> vs <code>ls --color=auto GNU</code> ; <code>date -v +1d BSD</code> vs <code>date -d GNU</code>).
<code>uname -a</code>	In thông tin hệ thống; chữ “Darwin” chứng minh Mac là hệ Unix, chuỗi “xnu-...” là tên nhân XNU.
<code>pwd</code>	In thư mục hiện hành (“tôi đang ở đâu”).
<code>ls</code>	Liệt kê file/thư mục (“ở đây có gì”).

Thuật ngữ	Định nghĩa 1 dòng
DevOps	Kết hợp phát triển + vận hành để đưa phần mềm ra nhanh, ổn, tự động; nền tảng là Linux.

12. Kế hoạch ôn ngắt quãng (Spaced Repetition)

Trí nhớ phai theo thời gian; ôn đúng nhịp sẽ “khóa” kiến thức lại. Hãy đánh dấu lịch:

- **Sau 1 ngày (ôn nhanh ~10 phút):** Mở Terminal, gõ lại từ trí nhớ cả 7 lệnh ở Lab 1 (`uname -a`, `sw_vers`, `whoami`, `date`, `pwd`, `ls`, `echo "..."`). Đọc lướt cột trái bộ Flashcards, tự nói định nghĩa.
- **Sau 3 ngày (ôn vừa ~20 phút):** Làm lại **toàn bộ phần Tự kiểm tra** mà KHÔNG nhìn đáp án. Chấm điểm bằng mục 13. Câu nào sai → đọc lại đúng mục đó.

Lưu ý

Tự kiểm tra chỉ có ở các mục **2, 3, 4, 5, 6, 7**. Các mục 8, 9, 10 là phần đọc hiểu, không có câu hỏi tự kiểm — bạn không bỏ sót gì cả nếu thấy chúng không có ô “Tự kiểm tra”.

- **Sau 1 tuần (ôn sâu ~30 phút):** Gập sách lại, tự viết ra giấy “**Tóm tắt 1 trang**” bằng lời của bạn, rồi so với mục 14. Làm thêm **Bài tập mở rộng** (mục 15). Nếu giải thích trôi chảy cho một người không biết gì → bạn đã nắm vững chương 1.

Mẹo

Ôn bằng cách **chủ động lôi kiến thức ra khỏi đầu** (tự gõ lệnh, tự trả lời) hiệu quả hơn nhiều so với đọc lại thụ động. Đây gọi là *active recall* — vũ khí học tập mạnh nhất.

13. Đáp án Tự kiểm tra

Mục 2 — Hệ điều hành

1. OS quản lý phần cứng (CPU, bộ nhớ, ổ cứng, thiết bị) và làm cầu nối để bạn và các ứng dụng dùng được máy mà không phải nói chuyện trực tiếp với phần cứng.
2. Ví dụ: Windows, macOS, Linux (hoặc Android, iOS). Máy bạn đang chạy **macOS**.
3. Từ dưới lên: **phần cứng** → **nhân (kernel)** hệ điều hành → **ứng dụng**.

Mục 3 — Lab lệnh

1. Lệnh `uname -a`; chữ **Darwin** (tên nhân macOS, một hệ Unix) là bằng chứng (và chuỗi `xnu-...` là tên đầy đủ của nhân XNU).
2. `sw_vers` — lệnh riêng của macOS, không có trên Linux.

3. `pwd` trả lời “tôi đang đứng ở thư mục nào” (vị trí); `ls` trả lời “trong thư mục này có những file/thư mục gì” (nội dung).

Mục 4 — Linux & GNU

1. **Linus Torvalds** viết nhân Linux năm **1991**. Dự án **GNU** do **Richard Stallman** khởi xướng năm **1983**.
2. Nói chính xác, “Linux” **chỉ là cái nhân (kernel)**, không phải toàn bộ hệ điều hành; phần còn lại chủ yếu là công cụ GNU (nên gọi đầy đủ là GNU/Linux).
3. “Tự do” = bạn có quyền xem/sửa/chia sẻ phần mềm (free as in *freedom*); “miễn phí” = giá bằng 0 (free as in *beer* — ai đó cho bạn lon bia không mất tiền). Linux có thể được bán có thu tiền (ví dụ dịch vụ Red Hat) mà vẫn là phần mềm tự do.

Mục 5 — Thuật ngữ nền tảng

1. **Terminal** = **cái điện thoại** (cửa sổ bạn gõ và nhìn); **shell** = **người đầu dây** thật sự hiểu lệnh và thực hiện (ví dụ `zsh`).
2. Distro = gói trọn bộ (nhân Linux + công cụ + ứng dụng). Ba ví dụ: **Ubuntu**, **Debian**, **Fedora**.
3. Vì nhân Linux trần không tự dùng được; cần distro ghép nó với công cụ và ứng dụng thành hệ thống hoàn chỉnh (ví dụ “động cơ” cần “khung xe”).

Mục 6 — Linux/macOS/Windows

1. macOS là **UNIX được chứng nhận chính thức**, do **The Open Group** cấp. **Linux** thuộc nhóm **Unix-like** (giống Unix nhưng không mang chứng nhận đó).
2. Nhân macOS tên đúng là **XNU**. **Mach KHÔNG phải** một nhánh của Unix — nó chỉ là một thành phần bên trong XNU.
3. Vì macOS dùng công cụ **BSD** còn Linux thường dùng công cụ **GNU**; một số flag (tùy chọn) khác nhau (ví dụ `ls -G` của BSD vs `ls --color=auto` của GNU; `date -v +1d` của BSD vs `date -d` của GNU), nên script viết theo cờ BSD có thể không chạy trên GNU và ngược lại.

Mục 7 — Lợi thế của Mac

1. Shell mặc định là **zsh**; kiểm chứng bằng `echo $SHELL` (kết quả `/bin/zsh` — đây là shell đăng nhập mặc định).
2. Bash trên Mac **cũ** (bản 3.2.57, dòng 3.2 phát hành năm 2006); kiểm chứng bằng `bash --version`.
3. Vì công cụ và **phiên bản** trên máy bạn (ví dụ bash 3.2, công cụ BSD) có thể khác với server (bash 5.x, công cụ GNU) → cùng một script có thể chạy khác nhau.

14. Tóm tắt 1 trang

- **Hệ điều hành (OS)** quản lý phần cứng và làm cầu nối giữa bạn với máy. Tư duy nền tảng: **phần cứng** → **nhân** → **ứng dụng** (tư duy theo lớp).
- **Linux** (về mặt chính xác) là **nhân (kernel)** do **Linus Torvalds** viết năm **1991**. Ghép với công cụ của dự án **GNU (Stallman, 1983)** thành hệ điều hành hoàn chỉnh (GNU/Linux).
- **Mã nguồn mở** = mã công khai, ai cũng xem/sửa/chia sẻ. “**Tự do**” ≠ “**miễn phí**”: free as in *freedom*, not *beer*.
- **6 thuật ngữ nền**: kernel (lõi OS), distro (gói trọn bộ: Ubuntu/Debian/Fedora), shell (bộ dịch lệnh, vd zsh), terminal (cửa sổ gõ lệnh), open source (mã mở), Unix-like (theo triết lý Unix).
- **macOS là UNIX chứng nhận chính thức** (The Open Group); nhân tên **XNU** (*Mach* chỉ là thành phần bên trong, KHÔNG phải nhánh Unix). **Linux là Unix-like**. → Dòng lệnh hai bên **rất giống nhưng không y hệt**: macOS dùng công cụ **BSD**, Linux dùng **GNU** → khác vài **flag** (vd `ls -G` vs `ls --color=auto`; `date -v +1d` vs `date -d`).
- **Lợi thế người dùng Mac**: Terminal có sẵn; shell mặc định **zsh** từ **Catalina (2019)** trở đi (máy mẫu đang chạy macOS 26.4.1); nhưng **bash trên Mac cũ** (3.2.57, dòng 3.2 từ 2006) → cẩn thận khi dùng tính năng bash mới. Lưu ý: server Linux thường dùng bash, nhưng lệnh cơ bản gần như giống nhau.
- **Bằng chứng Mac là Unix**: gõ `uname -a` → thấy **Darwin** (và chuỗi `xnu-...` = tên nhân XNU).
- **7 lệnh an toàn đã thực hành**: `uname -a`, `sw_vers`, `whoami`, `date`, `pwd`, `ls`, `echo "..."` (cộng `echo $SHELL`, `bash --version`).
- **Vì sao học Linux**: gần như mọi server, cloud và công cụ DevOps (Docker, Kubernetes, CI/CD) đều chạy Linux; kỹ năng dòng lệnh bền vững suốt sự nghiệp.
- **Tư duy senior xuyên suốt**: luôn hỏi “**vì sao**” và “**mình đang đứng trên nền nào**” (phiên bản, GNU vs BSD, lớp nào của hệ thống).

15. Bài tập mở rộng (Stretch)

Làm được càng nhiều càng tốt; không bắt buộc, nhưng đây là nơi bạn vượt lên. Nhắc lại: **mọi lệnh dưới đây đều an toàn, chỉ đọc thông tin**.

1. **Đọc tài liệu như senior**. Trong Terminal gõ:

Lệnh

```
man uname
```

`man` = *manual* (sổ tay hướng dẫn). Một trang hướng dẫn hiện ra. Dùng phím **mũi tên** để cuộn, tìm xem cờ `-a` mô tả ra sao, rồi bấm phím **q** để thoát. *Kỹ năng tự đọc*

man là một trong những kỹ năng quý nhất của nghề — *senior* tra *man* chứ không đoán mò.

Kết quả/đáp án gợi ý

Bạn sẽ thấy một trang văn bản; ở phần mô tả các cờ, `-a` được giải thích đại ý là “in ra tất cả thông tin” (*all*). Nếu màn hình kẹt ở chế độ đọc, cứ bấm `q` là thoát về dấu nhắc lệnh.

2. So sánh shell. Gõ:

Lệnh

```
cat /etc/shells
```

Lệnh này in danh sách các shell mà máy bạn biết. Tìm xem có cả `zsh` lẫn `bash` không. Tự hỏi: cái nào đang là mặc định của tôi (đã biết từ mục 7)?

Kết quả/đáp án gợi ý

Trên máy mẫu, danh sách điển hình gồm các dòng như `/bin/bash`, `/bin/zsh`, `/bin/sh`, `/bin/csh`, `/bin/tcsh`, `/bin/ksh`. Bạn sẽ thấy **cả `/bin/zsh` lẫn `/bin/bash`**; mặc định của bạn là `/bin/zsh` (đúng như `echo $SHELL` ở mục 7).

3. Tự “thăm vấn” máy của bạn. Lần lượt chạy `whoami`, `pwd`, `date`, `uname -a`. Viết ra giấy một câu cho mỗi lệnh: *lệnh này trả lời cho tôi câu hỏi gì?* (Đây là bài luyện diễn đạt — nói được tức là hiểu.)

Đáp án gợi ý

`whoami` → “tôi là user nào?”; `pwd` → “tôi đang ở thư mục nào?”; `date` → “bây giờ là mấy giờ, múi giờ nào?”; `uname -a` → “máy/nhân này là hệ gì, kiến trúc gì?”.

4. Săn sự khác biệt GNU vs BSD (nâng cao, chỉ đọc/quan sát). Trên Mac gõ:

Lệnh

```
date -v +1d
```

Trên máy mẫu BSD, cờ `-v` dùng để “cộng/trừ thời gian” (ví dụ `+1d` = cộng 1 ngày), nên lệnh in ra **ngày của hôm sau**. Cách làm tương đương trên Linux (GNU) là dùng `date -d`, cụ thể: `date -d "tomorrow"` hoặc `date -d "+1 day"`. **Tự rút ra:** cùng mục đích “tính ngày mai”, hai bên dùng cờ khác nhau — đúng như bài học GNU vs BSD ở mục 6.3.

Lưu ý

Đừng đem `date -v +1d` chạy trên Linux (nó sẽ báo lỗi vì GNU không có cờ `-v`), và ngược lại đừng đem `date -d ...` chạy trên Mac. Đây chính là cái bẫy “viết trên Mac, hỏng trên Linux” ở Góc Senior mục 6.3.

5. **Thử thách diễn đạt.** Tìm một người thân *không biết gì về máy tính* và giải thích cho họ trong 2 phút: “Linux là gì, và vì sao cái Mac của con/anh/chị thực ra cũng là một hệ Unix.” Nếu họ gật gù hiểu → bạn đã thật sự nắm chương 1.

Lời chúc cuối chương

Bạn vừa đi từ “chưa từng gõ lệnh nào” tới chỗ chứng minh được máy mình là hệ Unix, hiểu Linux từ gốc rễ, và phân biệt được nó với macOS/Windows. Đó là một bước đi thật, không hề nhỏ. Hẹn gặp lại ở **Chương 2**, nơi ta bắt đầu tự do dạo bước trong cây thư mục. Cứ gõ đều tay nhé — mỗi lệnh bạn gõ hôm nay là một viên gạch cho người senior bạn sẽ trở thành.